

Python Data Structures

Python Data Structures

Data structure = A container that stores data in a specific format.

Why do we need data structures?

To:

- ❑ Store multiple values
- ❑ Organize and retrieve data quickly
- ❑ Improve performance of programs
- ❑ Perform operations like searching, sorting, and updating data

these allow:

- ✓ Indexing (positive & negative)
- ✓ Slicing
- ✓ Iteration (loops)
- ✓ len() to find length
- ✓ Membership check (in, not in)

1 List 2 Tuple 3 Set 4 Dictionary

1. LIST IN PYTHON

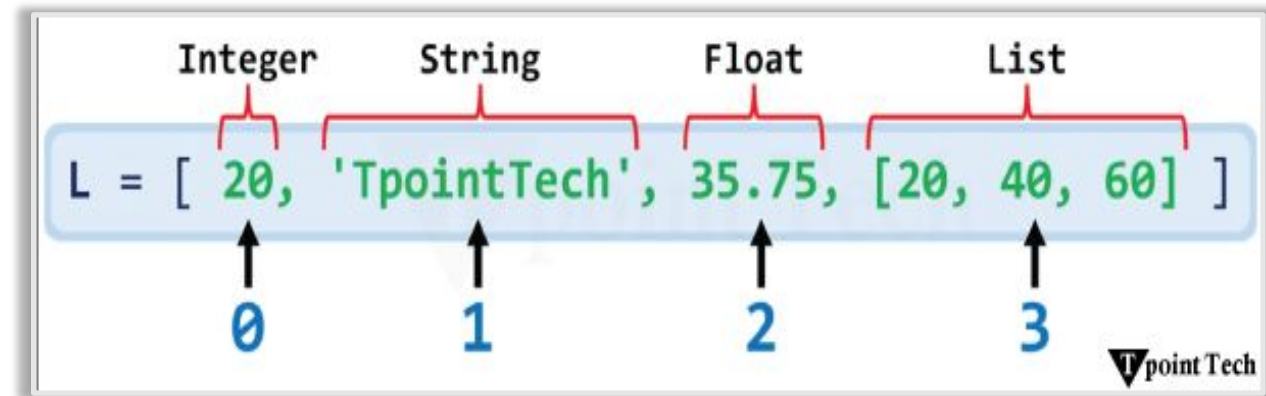
1. PYTHON LISTS

Built-in data structure that allows us to store multiple items in a single variable.

- ❑ Mutable
- ❑ Ordered
- ❑ Allows duplicate elements
- ❑ Can hold different type of elements
- ❑ Syntax:

```
List_name = [ item1,item2,item3,...]
```

```
Empty_list = [ ]
```



LIST OPERATORS

1. Concatenation Operator(+)

- Used to join two lists.
- Example:

$a = [1, 2]$

$b = [3, 4]$

$c = a + b \quad \# [1, 2, 3, 4]$

2. Repetition Operator(*)

- Repeats the list multiple times.
- Example:

$a = [1, 2]$

$b = a * 3 \quad \# [1, 2, 1, 2, 1, 2]$

LIST OPERATORS

3. Membership Operator(in, not in)

- Used to check if an item exists in a list.
- Example:

```
fruits = ["apple", "banana", "mango"]
```

```
"apple" in fruits    # True
```

```
"grape" not in fruits # True
```

LIST OPERATORS

3. List Indexing

-4	-3	-2	-1
fruits = ["apple", "banana", "mango", "grape"]			
0	1	2	3

fruits[1] = banana

fruits[-2] = mango

fruits[4] # ✗ Error (index out of range)

fruits[-5] # ✗ Error

LIST OPERATORS

4. List Slicing

extract a part of a list using [start : end : step], with the end index excluded.

Syntax: `list_name[start : end : step]`

Example: `fruits = ["apple", "banana", "mango", "grape", "orange"]`

`fruits[1:4]` # Output: ['banana', 'mango', 'grape']

`fruits[:3]` # ['apple', 'banana', 'mango']

`fruits[2:]` # ['mango', 'grape', 'orange']

`fruits[-4:-1]` # ['banana', 'mango', 'grape']

`fruits[0:5:2]` # ['apple', 'mango', 'orange']

LIST METHODS IN PYTHON

Method	Description	Example
append(x)	Adds item x to the end of the list	list.append(10)
insert(i, x)	Inserts x at index i	list.insert(2, "apple")
extend(iterable)	Adds all items from another list/iterable	list.extend([1, 2, 3])
remove(x)	Removes the first occurrence of x	list.remove("apple")
pop(i)	Removes and returns item at index i (default last)	list.pop(1)
clear()	Removes all items from the list	list.clear()
index(x)	Returns index of first occurrence of x	list.index("mango")
count(x)	Counts how many times x appears	list.count(5)
sort()	Sorts the list in ascending order	list.sort()
reverse()	Reverses the list order	list.reverse()
copy()	Returns a shallow copy of the list	new = list.copy()

BUILT-IN FUNCTIONS USED WITH LIST

Function	What it does	Example
len()	Finds number of items	len(list)
min()	Smallest value	min(list)
max()	Largest value	max(list)
sum()	Sum of values	sum(list)
sorted()	Returns a new sorted list	sorted(list)

NESTED LIST

□ A nested list is a list that contains one or more lists as its elements.

Example:

`n = [10, 20, [30, 40, 50], 60]`

<code>n[0]</code>	<code>n[1]</code>	<code>n[2][0]</code>	<code>n[2][1]</code>	<code>n[2][2]</code>	<code>n[3]</code>
<code>[10</code>	<code>, 20</code>	<code>, [30</code>	<code>, 40</code>	<code>, 50</code>	<code>], 60]</code>
<code>n[-4]</code>	<code>n[-3]</code>	<code>n[-2][-3]</code>	<code>n[-2][-2]</code>	<code>n[-2][-1]</code>	<code>n[-1]</code>

NESTED LIST

MATRIX

A **matrix** is a **rectangular arrangement of numbers** arranged in **rows** and **columns**.

In Python is a **list of lists**, where each inner list represents a **row**.

Example:

```
matrix = [  
    [1, 2, 3],    # Row 0  
    [4, 5, 6],    # Row 1  
    [7, 8, 9]     # Row 2  
]
```

NESTED LIST

MATRIX

```
matrix = [[1, 2, 3], [4, 5, 6],[7, 8, 9]]
```

```
for row in matrix:
```

```
    for item in row:
```

```
        print(item, end=" ")
```

```
    print()
```

```
#output:  1 2 3
```

```
          4 5 6
```

```
          7 8 9
```

Note:

□ Accessing Matrix Elements :

matrix[row][column]

□ Number of Rows & Columns :

rows = len(matrix)

cols = len(matrix[0])

LIST COMPREHENSION

Question 1: Create a list that contains the squares of numbers from 1-10.

Using for loop:

```
squares = []  
for i in range(1, 6):  
    squares.append(i * i)
```

Using List comprehension

```
squares = [i*i for i in range(1, 6)]
```

LIST COMPREHENSION

List comprehension is a short and easy way to create a new list from an existing sequence using a single line of code.

Syntax:

[expression for item in iterable if condition]

Example:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

```
new_list = [x for x in fruits if "a" in x]
```

```
# Output: ['apple', 'banana', 'mango']
```

LIST COMPREHENSION

when to use list comprehension?

- ❑ to replace simple for-loops that build lists by appending elements.
- ❑ to include conditional filtering or conditional transformations within a single expression.
- ❑ When your logic is straightforward enough to keep the comprehension readable and maintainable.
- ❑ When you aim for code clarity and efficiency.